

Community Toolkits for .NET

Documentation

The home for Community Toolkits for .NET which are partnering the community with engineers, Microsoft MVPs, and the teams working on those technologies. The Community Toolkits offer a collection of reusable implementations that aim to simplify common development tasks for a variety of .NET frameworks.

.NET Community Toolkit

OVERVIEW

[What is the .NET Community Toolkit?](#)

REFERENCE

[Introduction to the MVVM Toolkit](#)

[Introduction to the Diagnostics Package](#)

[Introduction to the High Performance Package](#)

GET STARTED

[Installation](#)

Windows Community Toolkit

OVERVIEW

[What is the Windows Community Toolkit?](#)

QUICKSTART

[Windows Community Toolkit Gallery !\[\]\(3342c215b2a8b663596a81468d5dc314_img.jpg\)](#)

REFERENCE

[Graph Helpers and Controls](#)

GET STARTED

[Installation](#)

[Using with Uno Platform](#) 

.NET Multi-platform App UI (.NET MAUI) Community Toolkit

OVERVIEW

[What is the .NET MAUI Community Toolkit?](#)

GET STARTED

[Installation](#)

CONCEPT

[Alerts](#)

[Animations](#)

[Behaviors](#)

[Converters](#)

[Essentials](#)

[Extensions](#)

[Image Sources](#)

[Layouts](#)

[Views](#)

[C# Markup](#)

.NET MAUI Native Library Interop

OVERVIEW

[What is the Native Library Interop?](#)

GET STARTED

.NET Community Toolkit Introduction

Article • 05/24/2022

[.NET Community Toolkit](#) is a collection of helpers and APIs that work for all [.NET developers](#) and are agnostic of any specific UI platform. The toolkit is maintained and published by Microsoft, and part of the [.NET Foundation](#).

The .NET Community Toolkit is available as a set of NuGet Packages for new or existing .NET projects.

The toolkit contains .NET Standard libraries (originally developed as part of the [Windows Community Toolkit](#)) that can be used both by application developers (regardless on the specific UI framework in use) and library authors. These libraries are also being used internally at Microsoft to power many of our first party apps (such as the new Microsoft Store) and constantly improved by listening to feedbacks from other teams, external partners, and other developers from the community. Here's a quick breakdown of the various components you'll find:

- [CommunityToolkit.Mvvm \(aka MVVM Toolkit\)](#): a fast, modular, platform-agnostic MVVM library, which is the official successor of [MvvmLight](#). It's used extensively in the Microsoft Store and other first party apps.
- `CommunityToolkit.Mvvm.SourceGenerators`: the source generators to augment the MVVM Toolkit.
- [CommunityToolkit.Diagnostics](#): a set of helper APIs (specifically, Guard and ThrowHelper) that can be used for cleaner, more efficient and less error-prone argument validation and error checking.
- [CommunityToolkit.HighPerformance](#) a collection of helpers for working in high-performance scenarios. It includes APIs such as pooled buffer helpers, a fast string pool type, a 2D variant of `Memory<T>` and `Span<T>` ([Memory2D<T>](#) and [Span2D<T>](#)) also supporting discontinuous regions, helpers for bit shift operations (such as `BitHelper`, also used in [Paint.NET](#)), and more.
- `CommunityToolkit.Common`: a set of helper APIs shared with other CommunityToolkit libraries.

You can also preview the capabilities of the [MVVM Toolkit](#) by running the [sample app available here](#).

Feel free to browse the documentation using the table of contents on the left side of this page.

Get started

Follow the [Getting started guide](#) for more detailed information about using the toolkit.

Open source

The .NET Community Toolkit is an open source project hosted on GitHub by the community as part of the [.NET Foundation](#) [↗]:

- [.NET Community Toolkit repo](#) [↗]
- [MVVM Toolkit Samples repo](#) [↗]

Getting Started with the Windows Community Toolkit

Article • 06/18/2021

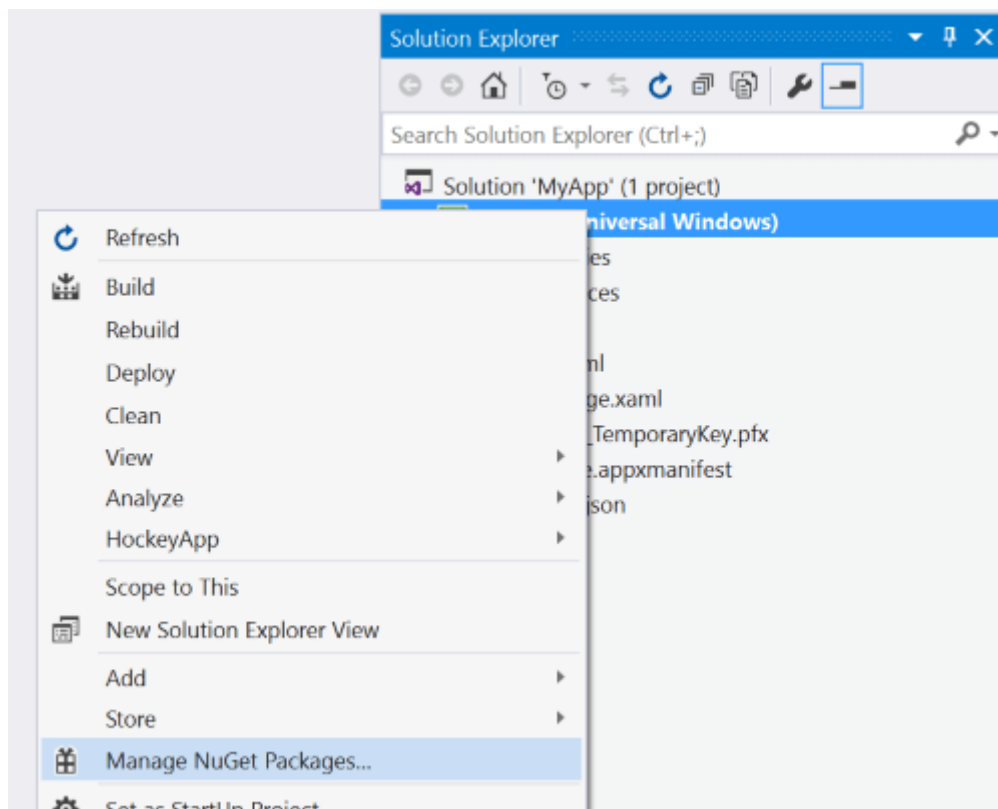
The toolkit is available as NuGet packages that can be added to any existing or new project using Visual Studio.

1. Download [Visual Studio 2017](#) and ensure you choose the **Universal Windows Platform development** Workload in the Visual Studio installer.

⚠ Note

Visual Studio 2015 is not supported with the Windows Community Toolkit 2.0 or higher

2. Open an existing project, or create a new project using the Blank App template under Visual C# -> Windows -> Universal. **Important:** Version 1809 (Build 17763) or higher is supported by current version of the Toolkit.
3. In Solution Explorer panel, right click on your project name and select **Manage NuGet Packages**. Search for **Microsoft.Toolkit.UWP**, and choose your desired [NuGet Packages](#) from the list.



4. Add a reference to the toolkit in your XAML pages or C#

- In your XAML page, add a reference at the top of your page:

XAML

```
xmlns:controls="using:Microsoft.Toolkit.Uwp.UI.Controls"
```

- In your C# page, add the namespaces to the toolkit:

C#

```
using Microsoft.Toolkit.Uwp;
```

5. You can copy and paste code snippets for each feature from the [Windows Community Toolkit Sample App](#).

Other Resources

Download the [Windows Community Toolkit Sample App](#) from the Windows store to see the controls in an actual app.

We recommend developers who are new to UWP to visit the [Getting Started with UWP Development](#) pages on the Developer portal.

Visit the [Windows Community Toolkit Github Repository](#) to see the current source code, what is coming next, and clone the repository. Community contributions are welcome!

Introduction to the MVVM Toolkit

Article • 11/07/2024

The `CommunityToolkit.Mvvm` package (aka MVVM Toolkit, formerly named `Microsoft.Toolkit.Mvvm`) is a modern, fast, and modular MVVM library. It is part of the .NET Community Toolkit and is built around the following principles:

- **Platform and Runtime Independent** - .NET Standard 2.0, .NET Standard 2.1 and .NET 6 🚀 (UI Framework Agnostic)
- **Simple to pick-up and use** - No strict requirements on Application structure or coding-paradigms (outside of 'MVVM'ness), i.e., flexible usage.
- **À la carte** - Freedom to choose which components to use.
- **Reference Implementation** - Lean and performant, providing implementations for interfaces that are included in the Base Class Library, but lack concrete types to use them directly.

The MVVM Toolkit is maintained and published by Microsoft, and part of the .NET Foundation. It is also used by several first party applications that are built into Windows, such as [the Microsoft Store](#) 🖱️.

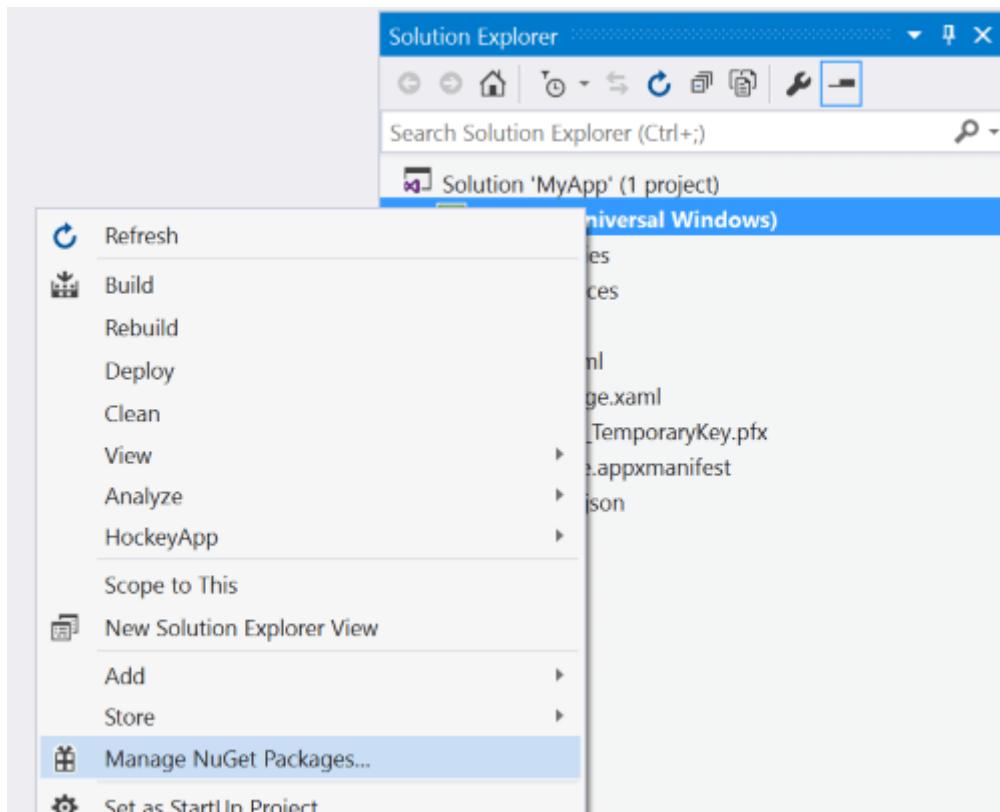
This package targets **.NET Standard** so it can be used on any app platform: WinUI 3, UWP, WinForms, WPF, Xamarin, Uno, and more; and on any runtime: .NET Native, .NET Core, .NET Framework, or Mono. It runs on all of them. The API surface is identical in all cases, making it perfect for building shared libraries.

Additionally, the MVVM Toolkit also has a **.NET 6** target, which is used to enable more internal optimizations when running on .NET 6. The public API surface is identical in both cases, so NuGet will always resolve the best possible version of the package without consumers having to worry about which APIs will be available on their platform.

Getting started

To install the package from within Visual Studio:

1. In Solution Explorer, right-click on the project and select **Manage NuGet Packages**. Search for **CommunityToolkit.Mvvm** and install it.



2. Add a using or Imports directive to use the new APIs:

c#

```
using CommunityToolkit.Mvvm;
```

VB

```
Imports CommunityToolkit.Mvvm
```

3. Code samples are available in the other docs pages for the MVVM Toolkit, and in the [unit tests](#) for the project.

When should I use this package?

Use this package for access to a collection of standard, self-contained, lightweight types that provide a starting implementation for building modern apps using the MVVM pattern. These types alone are usually enough for many users to build apps without needing additional external references.

The included types are:

- **CommunityToolkit.Mvvm.ComponentModel**
 - [ObservableObject](#)
 - [ObservableRecipient](#)

- [ObservableValidator](#)
- **CommunityToolkit.Mvvm.DependencyInjection**
 - [Ioc](#)
- **CommunityToolkit.Mvvm.Input**
 - [RelayCommand](#)
 - [RelayCommand<T>](#)
 - [AsyncRelayCommand](#)
 - [AsyncRelayCommand<T>](#)
 - [IRelayCommand](#)
 - [IRelayCommand<T>](#)
 - [IAsyncRelayCommand](#)
 - [IAsyncRelayCommand<T>](#)
- **CommunityToolkit.Mvvm.Messaging**
 - [IMessenger](#)
 - [WeakReferenceMessenger](#)
 - [StrongReferenceMessenger](#)
 - [IRecipient<TMessage>](#)
 - [MessageHandler<TRecipient, TMessage>](#)
- **CommunityToolkit.Mvvm.Messaging.Messages**
 - [PropertyChangedMessage<T>](#)
 - [RequestMessage<T>](#)
 - [AsyncRequestMessage<T>](#)
 - [CollectionRequestMessage<T>](#)
 - [AsyncCollectionRequestMessage<T>](#)
 - [ValueChangedMessage<T>](#)

This package aims to offer as much flexibility as possible, so developers are free to choose which components to use. All types are loosely-coupled, so that it's only necessary to include what you use. There is no requirement to go "all-in" with a specific series of all-encompassing APIs, nor is there a set of mandatory patterns that need to be followed when building apps using these helpers. Combine these building blocks in a way that best fits your needs.

Additional resources

- Check out the [sample app](#)  (for multiple UI frameworks) to see the MVVM Toolkit in action.
- You can also find more examples in the [unit tests](#) .

Introduction to the Diagnostics package

Article • 11/07/2024

The `CommunityToolkit.Diagnostics` package contains APIs to efficiently validate method parameters and to throw exceptions in faulting code paths. It is meant to be used to help simplify all argument checks and to make them more expressive and easier to read, while at the same time improving codegen quality and performance.

This package can be installed through NuGet, and it has the following multi-targets:

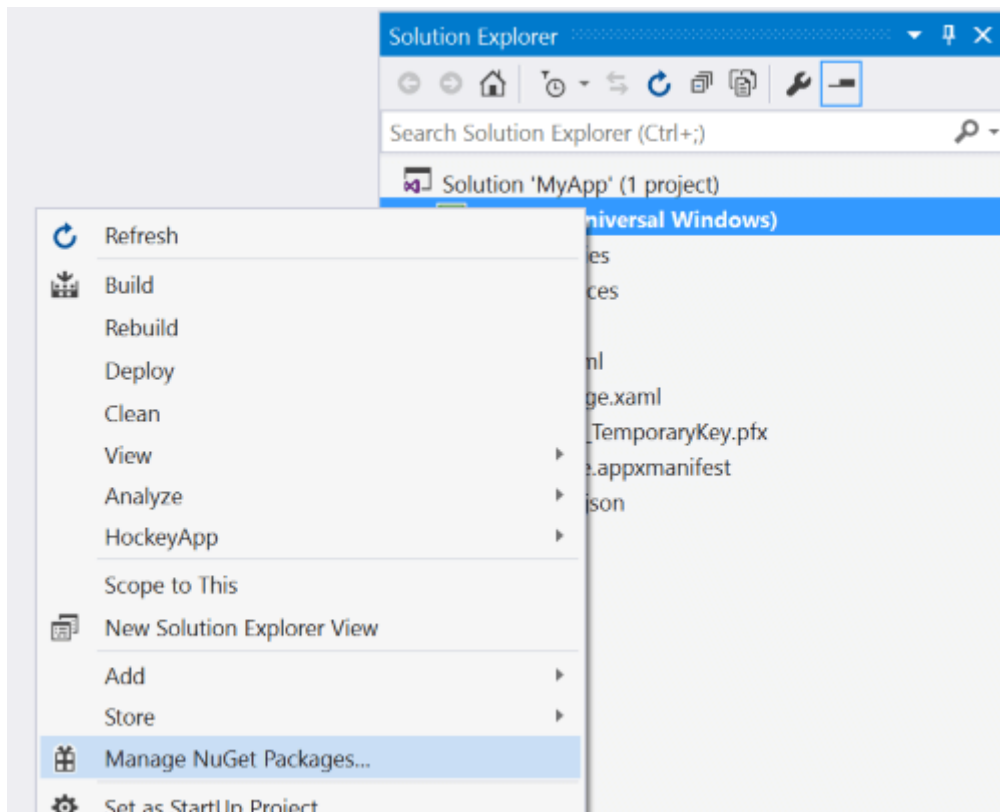
- .NET Standard 2.0
- .NET Standard 2.1
- .NET 6

This means the package can be used on any available runtime (including .NET Framework, .NET Core, UWP, Unity, Xamarin, Uno, Blazor, etc.). The API surface is almost identical in all cases, while the internal implementation can be optimized when newer APIs are available. The Diagnostics package as a whole is meant to be self-contained and extremely small in scope and binary size.

Getting started

To install the package from within Visual Studio:

1. In Solution Explorer, right-click on the project and select **Manage NuGet Packages**. Search for `CommunityToolkit.Diagnostics` and install it.



2. Add a using or Imports directive to use the new APIs:

c#

```
using CommunityToolkit.Diagnostics;
```

VB

```
Imports CommunityToolkit.Diagnostics
```

Additional resources

You can find more examples in the [unit tests](#).

Guard

Article • 11/07/2024

The [Guard](#) can be used to validate method arguments in a streamlined manner, which is also faster, less verbose, more expressive and less error prone than manually writing checks and throwing exceptions.

Platform APIs: [Guard](#), [CallerArgumentExpressionAttribute](#)

How it works

These `Guard` APIs are built with three core principles in mind:

- Being **fast**. To achieve that, all the APIs have been designed to produce as little code as possible in the caller, and each single Guard API will (almost always) be inlined. Furthermore, specialized methods are generated with T4 templates to achieve the most efficient assembly code possible when working with common types (eg. primitive numeric types).
- Being **helpful**. Each `Guard` API produces a detailed exception message that clearly specifies what went wrong, along with additional info (eg. current variable values), when applicable.
- Being **intuitive**. To achieve this, all the `Guard` APIs have expressive and self-explanatory names that make it immediately clear what each API is supposed to do. This shifts the burden of writing checks away from the developers, letting them express conditions using natural language.

Syntax

Here is a sample method, with some checks being done with explicitly and with manual throw statements:

C#

```
public static void SampleMethod(int[] array, int index, Span<int> span,
string text)
{
    if (array is null)
    {
        throw new ArgumentNullException(nameof(array), "The array must not
be null");
    }
}
```

```

    if (array.Length >= 10)
    {
        throw new ArgumentException($"The array must have less than 10 items, had a size of {array.Length}", nameof(array));
    }

    if (index < 0 || index >= array.Length)
    {
        throw new ArgumentOutOfRangeException(nameof(index), $"The index must be in the [0, {array.Length}) range, was {index}");
    }

    if (span.Length < array.Length)
    {
        throw new ArgumentException($"The target span is shorter than the input array, had a length of {span.Length}", nameof(span));
    }

    if (string.IsNullOrEmpty(text))
    {
        throw new ArgumentException("The input text can't be null or empty", nameof(text));
    }
}

```

And here is the same method, but using the new `Guard.APIs` to validate the input arguments:

```

C#

public static void SampleMethod(int[] array, int index, Span<int> span, string text)
{
    Guard.IsNotNull(array);
    Guard.HasSizeGreaterThanOrEqualTo(array, 10);
    Guard.IsInRangeFor(index, array);
    Guard.HasSizeLessThanOrEqualTo(array, span);
    Guard.IsNotNullOrEmpty(text);
}

```

The `Guard` APIs will perform the required checks in the fastest way possible, and will throw the appropriate exception with a well formatted message if they fail.

❗ Note

The `Guard` APIs rely on [\[CallerArgumentExpression\]](#) to automatically infer the name of the argument being validated. This requires C# 10 to be enabled in the project in use. If you're using a lower version of the language, you will need to

manually pass the parameter name, for instance using `nameof()` to refer to it (eg. `Guard.IsNotNull(array, nameof(array))`).

Methods

There are dozens of different APIs and overloads in the `Guard` class, here are a few of them:

General

[Expand table](#)

Methods	Return Type	Description
<code>IsNotNull<T>(T, string)</code>	<code>void</code>	Asserts that the input value is not null
<code>IsOfType<T>(object, string)</code>	<code>void</code>	Asserts that the input value is of a specific type
<code>IsAssignableToType<T>(object, string)</code>	<code>void</code>	Asserts that the input value can be assigned to a specified type
<code>IsReferenceEqualTo<T>(T, T, string)</code>	<code>void</code>	Asserts that the input value must be the same instance as the target value
<code>IsTrue(bool, string)</code>	<code>void</code>	Asserts that the input value must be true

Comparisons

[Expand table](#)

Methods	Return Type	Description
<code>IsEqualTo<T>(T, T, string)</code>	<code>void</code>	Asserts that the input value must be equal to a specified value
<code>IsBitwiseEqualTo<T>(T, T, string)</code>	<code>void</code>	Asserts that the input value must be a bitwise match with a specified value
<code>IsLessThan<T>(T, T, string)</code>	<code>void</code>	Asserts that the input value must be less than a specified value
<code>IsLessThanOrEqualTo<T>(T, T, string)</code>	<code>void</code>	Asserts that the input value must be less than or equal to a specified value

Methods	Return Type	Description
IsInRange<T>(T, T, T, string)	void	Asserts that the input value must be in the [minimum, maximum) range
IsBetween<T>(T, T, T, string name)	void	Asserts that the input value must be in the (minimum, maximum) interval
IsBetweenOrEqualTo<T>(T, T, T, string name)	void	Asserts that the input value must be in the [minimum, maximum] interval

Strings

[Expand table](#)

Methods	Return Type	Description
IsNotNullOrEmpty(string, string)	void	Asserts that the input string instance must not be null or empty
IsNotNullOrWhiteSpace(string, string)	void	Asserts that the input string instance must not be null or whitespace

Collections

[Expand table](#)

Methods	Return Type	Description
IsNotEmpty<T>(T[], string)	void	Asserts that the input array instance must not be empty
HasSizeEqualTo<T>(T[], int, string)	void	Asserts that the input array instance must have a size of a specified value
HasSizeAtLeast<T>(T[], int, string)	void	Asserts that the input array must have a size of at least or equal to a specified value
IsInRangeFor<T>(int, T[], string)	void	Asserts that the input index is valid for a given array
HasSizeLessThanOrEqualTo<T>(T[], T[], string)	void	Asserts that the source array must have a size of less than or equal to that of the destination array

Tasks

 Expand table

Methods	Return Type	Description
IsCompleted(Task, string)	void	Asserts that the input task is in a completed state
IsNotCanceled(Task, string)	void	Asserts that the input task is not canceled

Examples

You can find more examples in the [unit tests](#).

ThrowHelper

Article • 11/07/2024

The [ThrowHelper](#) class is a helper type that can be used to efficiently throw exceptions. It is meant to support the [Guard](#) APIs, and it should primarily be used in situations where developers need fine-grained controls over the exception types being thrown, or over the exact exception message to include.

Platform APIs: [ThrowHelper](#), [Guard](#)

Syntax

The `ThrowHelper` class exposes a series of `ThrowException` APIs that offer a 1:1 mapping to the constructors of all the available exception types. These methods are meant to be used in place of the classic `throw new Exception(...)` statement in code, as it will result in smaller and more efficient code. Put simply:

C#

```
// Replace this...
throw new InvalidOperationException("Some custom message from my library");

// ...with this
ThrowHelper.ThrowInvalidOperationException("Some custom message from my library");
```

⚠ Note

This will also result in a slightly different stack trace, as the exception will be thrown by this helper method instead of directly by the original caller. The practical result is still the same as throwing an exception directly.

The `Guard` APIs provide an easy to use abstraction over many common checks that might need to be performed, so it is recommended to first try to see if there is a `Guard` API that can be used, and only fallback on using `ThrowHelper` if that's not the case. As mentioned before, this might be if your library needs to throw a specific exception type not present in the `Guard` APIs, or if you need full control on the exact arguments being included in the exception being thrown.

There are also generic overloads for all the available APIs, which can be used within expressions that require a return type of a specific type (eg. a switch expression). These overloads never return any value, but the return type is declared to allow the C# compiler to accept the usage of these APIs within expressions that wouldn't work with a method returning `void`. Consider the following example:

C#

```
int result = text switch
{
    "cat" => 0,
    "dog" => 1,
    _ => ThrowHelper.ThrowArgumentException<int>(nameof(text))
};
```

Here we're using `ThrowHelper` within an expression that requires a return type of type `int`, so we can use the generic overload of `ThrowArgumentException` to make this possible. This also works with patterns such as ternary operators (`x ? a : b`), null-coalescing operators (`x = a ?? b`) null-coalescing assignment operators (`x ??= y`), and more.

Methods

There are lots of different methods and overloads in the `ThrowHelper` class, providing access to the following exceptions (and mapping all their public constructors):

- [ArrayTypeMismatchException](#)
- [ArgumentException](#)
- [ArgumentNullException](#)
- [ArgumentOutOfRangeException](#)
- [COMException](#)
- [ExternalException](#)
- [FormatException](#)
- [InsufficientMemoryException](#)
- [InvalidDataException](#)
- [InvalidOperationException](#)
- [LockRecursionException](#)
- [MissingFieldException](#)
- [MissingMemberException](#)
- [MissingMethodException](#)
- [NotSupportedException](#)
- [ObjectDisposedException](#)

- [OperationCanceledException](#)
- [PlatformNotSupportedException](#)
- [SynchronizationLockException](#)
- [TimeoutException](#)
- [UnauthorizedAccessException](#)
- [Win32Exception](#)

You can see the available constructors for each of these exception types from the linked docs - the respective `ThrowHelper` APIs will simply offer a series of overloads corresponding to the existing constructors for that specified type. Each API is simply named "Throw" + the exception type.

Technical Details

The following section describes the impact of switching to `ThrowHelper` APIs over the standard `throw new Expression` statements with respect to the codegen. As mentioned above, knowing these details is not necessary in order to use these APIs, but for developers familiar with how the runtime works or with how the JIT processes code during execution, this will provide a clearer explanation of why using the `ThrowHelper` pattern results in more compact and faster code. In order to do so, we will need to have a peek at the code that the JIT compiler is actually producing in these situations. We will use the code generated on .NET Core 3.1 x64 as a reference, but the same concept applies to other runtimes and architectures as well.

Let's consider this simple type:

C#

```
public struct ExceptionTest
{
    private int number;

    public int GetValueIfNotZeroOrThrow()
    {
        if (number == 0)
        {
            throw new InvalidOperationException("The value must be != 0");
        }

        return number;
    }
}
```

This type is only meant to showcase the use of `ThrowHelper` APIs. Here we're simply checking the value of the `number` field, and throwing if it's 0. Let's check the generated asm code:

x86asm

```
ExceptionTest.GetValueIfNotZeroOrThrow()  
    mov eax, [rcx]           ; read number  
    test eax, eax           ; if (number != 0)  
    je short L0011          ; jump to faulting path (if == 0)  
    ret                     ; return (number is already in eax)  
    mov rcx, 0x7ff95cbaca80  ; exception setup...  
    call 0x00007ff9bc6178f0  
    mov rsi, rax  
    mov ecx, 1  
    mov rdx, 0x7ff96590c080  
    call 0x00007ff9bc7403e0  
    mov rdx, rax  
    mov rcx, rsi  
    call System.InvalidOperationException..ctor(System.String)  
    mov rcx, rsi  
    call 0x00007ff9bc5db3a0  
    int3 ; finally throw the exception
```

We can see the code contains a lot of lines just dedicated to creating the exception being thrown. This can cause a large increase in the code size whenever we have many of these checks because it will reduce the efficiency of the instruction cache, etc. In general, we'd like the size of our methods to be as small as possible.

The above snippet, rewritten using the `ThrowHelper` APIs, would look like this:

C#

```
public int GetValueIfNotZeroOrThrow()  
{  
    if (number == 0)  
    {  
        ThrowHelper.ThrowInvalidOperationException("The value must be !=  
0");  
    }  
  
    return number;  
}
```

Which results in the following assembly:

x86asm

```

ExceptionTest.GetValueIfNotZeroOrThrow2()
    mov eax, [rcx]           ; load number
    test eax, eax           ; if (number != 0)
    je short L000f          ; faulting path (if == 0)
    ret                     ; return number (already in eax)
    mov rcx, 0x23435d85b98   ; load the exception message
    mov rcx, [rcx]
    call ThrowHelper.ThrowInvalidOperationException(System.String) ;
ThrowHelper call!
    int3 ; return with fault

```

We can see that the resulting assembly is much smaller than before. We basically moved all the code to create the exception out of the method, which also means we can just always reuse the same code from the `ThrowHelper` APIs, with different arguments, instead of inlining the exception throw in every single one of our methods. Here we only have those two `mov` instructions to load the `string` with our exception type, and then we jump to the `ThrowHelper.ThrowInvalidOperationException` method. The JIT compiler is able to see that that method will always throw, so it will never inline that call, and it will make sure to rewrite our conditional branches in the best way possible (specifically, knowing what is the branch that is supposed to be executed, without faulting).

Examples

You can find more examples in the [unit tests](#) [↗](#).

Introduction to the High Performance package

Article • 11/07/2024

The `CommunityToolkit.HighPerformance` contains helpers and extensions to work in high-performance scenarios. This package can be installed through NuGet, and it has the following multi-targets:

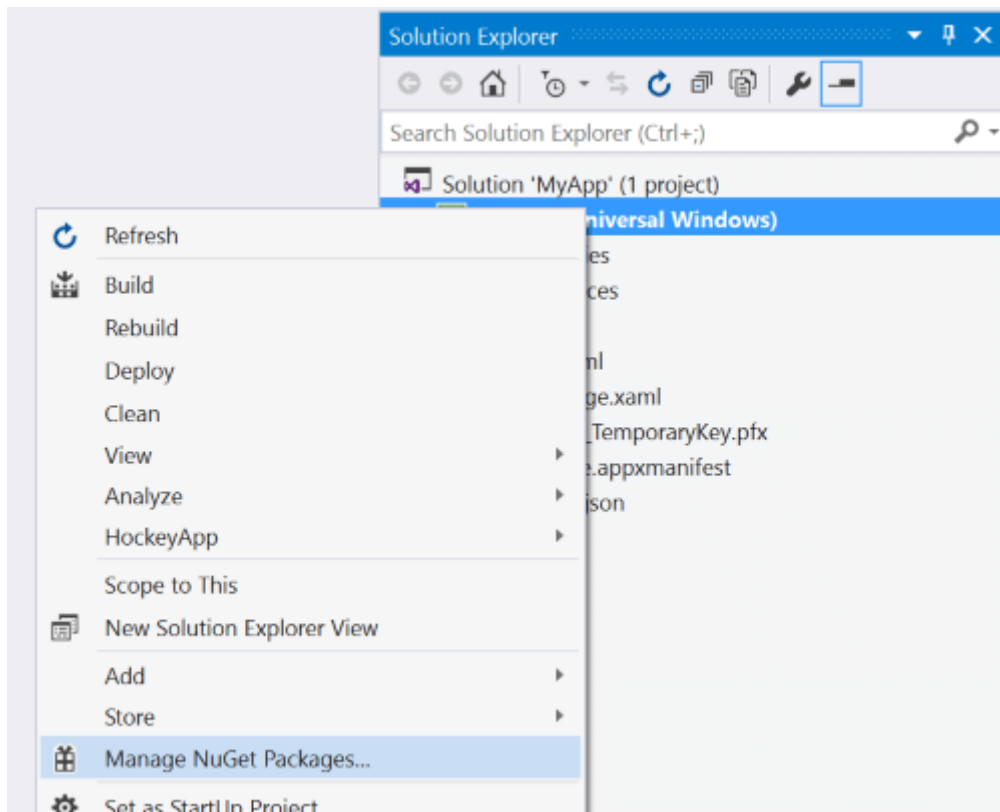
- .NET Standard 2.0
- .NET Standard 2.1
- .NET 6
- .NET 7

This means that you can use it from anything from UWP or legacy .NET Framework applications, games written in Unity, cross-platform mobile applications using Xamarin, to .NET Standard libraries and modern .NET 6 and .NET 7 applications. The API surface is almost identical in all cases, and lots of work has been put into backporting as many features as possible to older targets like .NET Standard 2.0. Except for some minor differences, you can expect the same APIs to be available on all target frameworks. The reason why multi-targeting has been used is to allow the package to leverage all the latest APIs on modern runtimes (like .NET 7) whenever possible, while still offering most of its functionalities to all target platforms.

Getting started

To install the package from within Visual Studio:

1. In Solution Explorer, right-click on the project and select **Manage NuGet Packages**. Search for **CommunityToolkit.HighPerformance** and install it.



2. Add a using or Imports directive to use the new APIs:

c#

```
using CommunityToolkit.HighPerformance;
```

VB

```
Imports CommunityToolkit.HighPerformance
```

3. Code samples are available in the other docs pages for the MVVM Toolkit, and in the [unit tests](#) for the project.

When should I use this package?

As the name suggests, the High Performance package contains a set of APIs that are heavily focused on optimization. All the new APIs have been carefully crafted to achieve the best possible performance when using them, either through reduced memory allocation, micro-optimizations at the assembly level, or by structuring the APIs in a way that facilitates writing performance oriented code in general.

This package makes heavy use of APIs such as:

- [System.Span<T>](#)
- [System.Memory<T>](#)

- [System.Buffers.ArrayPool<T>](#)
- [System.Runtime.CompilerServices.Unsafe](#)
- [System.Runtime.InteropServices.MemoryMarshal](#)
- [System.Threading.Tasks.Parallel](#)

If you are already familiar with these APIs or even if you're just getting started with writing high performance code in C# and want a set of well tested helpers to use in your own projects, have a look at what's included in this package to see how you can use it in your own projects!

Where to start?

Here are some APIs you could look at first, if you were already using one of those types mentioned above:

- [Span2D<T>](#) and [Memory2D<T>](#), for a `Span<T>` and `Memory<T>`-like abstraction over 2D memory
- [MemoryOwner<T>](#) and [SpanOwner<T>](#), if you were using `System.Buffers.ArrayPool<T>`.
- [StringPool](#), for an `ArrayPool<T>`-like type to cache `string` instances
- [ParallelHelper](#), if you were using `System.Threading.Tasks.Parallel`.

Additional resources

You can find more examples in the [unit tests](#) [↗](#).

Span2D<T>

Article • 11/07/2024

The [Span2D<T>](#) is a type that mirrors the functionality of the [Span<T>](#) type, but it supports 2D memory regions. Just like [Memory2D<T>](#), it is extremely flexible and can wrap a number of different objects, as well as native pointers or GC references.

The internal layout is similar to that used by the [Memory2D<T>](#) type, including a pitch parameter that is used to enable support for discontinuous memory buffers. You can read more info on this in the [Memory2D<T>](#) docs.

Platform APIs: [Span2D<T>](#), [Memory2D<T>](#), [ReadOnlySpan2D<T>](#)

Syntax

Here's how you can create a [Span2D<T>](#) instance from a 2D array:

C#

```
int[,] array =
{
    { 1, 2, 3 },
    { 4, 5, 6 }
};

Span2D<int> span = array;

// The memory directly maps the 2*3 array here

span[0, 0] = 10;
span[2, 1] = 20;

// The array is now:
// { 10, 2, 3 },
// { 4, 20, 6 }

// We can also use indices, on supported runtimes
int x = span[0, ^1];

// We can also get references to items, like with arrays
ref int reference = ref span[1, 1];

Span2D<int> slice = span.Slice(0, 1, 2, 2);

// Or alternatively, on supported runtimes

slice = span[.., 1..];
```

```
int[,] copy = slice.ToArray();

// The resulting array is:
// { 2, 3 },
// { 20, 6 }
```

We can also directly create a 2D view over native memory:

```
C#

int* p = stackalloc int[9];

Span2D<int> span = new Span2D<int>(p, 3, 3);
```

The `Span2D<T>` type also includes custom enumerator types to easily traverse a given row, column or the entire memory area using the standard `foreach` syntax in C#, as well as performing bulk operations in a single call:

```
C#

int[,] array =
{
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};

Span2D<int> span = array;

foreach (int i in span.GetColumn(1))
{
    // 2, 5, 8
}

// We can also iterate by reference
foreach (ref int i in span.GetRow(2))
{
    // 7, 8, 9
}

foreach (int i in span)
{
    // 1, 2, 3, 4, 5...
}

// Set all items in a column to 0
span.GetColumn(0).Clear();

// Set the value of all items in a row
span.GetRow(1).Fill(42);
```

```
Span<int> copy = stackalloc int[3];

// Copy all items from a column
span.GetColumn(2).CopyTo(copy);

// Get a copy of a row as an array
int[] array = span.GetRow(1).ToArray();
```

ReadOnlySpan2D<T>

The `ReadOnlySpan2D<T>` is to the `Span2D<T>` type what `ReadOnlySpan<T>` is to `Span<T>`.

It exposes a similar set of APIs but provides no way to directly modify the contents of the underlying memory area.

Sample Code

You can find more examples in the [unit tests](#).

Memory2D<T>

Article • 11/07/2024

The `Memory2D<T>` is a type that mirrors the functionality of the `Memory<T>` type, with the difference being that it can be used to represent 2D memory locations. It is extremely flexible and is capable of wrapping a number of different types, including ND arrays (with explicit support for 1D, 2D, and 3D arrays) or `Memory<T>` instances. This type is meant to be used together with the `Span2D<T>` type, in the same way that `Memory<T>` is used along with `Span<T>`. For more info on the key differences and use case scenarios of these two types, you can read [this docs page](#).

Platform APIs: `Memory2D<T>`, `Span2D<T>`, `ReadOnlyMemory2D<T>`

How it works

The `Memory2D<T>` type internally tracks the mapped 2D memory area through a reference to the wrapped object, the height and width parameters, and a special pitch parameter. The height and width indicate the length of the rows and columns in the 2D memory area, while the pitch indicates the offset between the end of each row and the start of the following one.

Here's a simple diagram that illustrates this configuration (the "XX" cells in the grid represent items belonging to the target 2D memory area):

C#

```
//                                     stride_...
// reference_ / width_ ...
//          \ /          \ /
// | -- | -- | - | -- | -- | -- | -- | -- | -- | _
// | -- | -- | XX | XX | XX | XX | XX | XX | -- | -- | |
// | -- | -- | XX | XX | XX | XX | XX | XX | -- | -- | |
// | -- | -- | XX | XX | XX | XX | XX | XX | -- | -- | |_height
// | -- | -- | XX | XX | XX | XX | XX | XX | -- | -- | _|
// | -- | -- | -- | -- | -- | -- | -- | -- | -- | -- |
// | -- | -- | -- | -- | -- | -- | -- | -- | -- | -- |
// ...__pitch_/
// ..._____/
```

This configuration allows `Memory2D<T>` to be extremely flexible in the way it maps existing buffers to 2D memory areas, as it makes it possible to also represent discontinuous buffers as a "virtual" 2D memory location. For instance, here's a few examples of buffer types that a `Memory2D` instance can map to:

- A 1D `T[]` array which is mapped as a 2D memory area in row-major order.
- A 2D `T[,]` array, mapped directly to a `Memory2D<T>` instance.
- A 3D `T[,,]` array, with a `Memory2D<T>` instance representing a given depth slice (a layer).

The `Memory<T>` type also exposes a number of utility methods, including most of the same API surface that the standard `Memory<T>` implements. For instance, it includes a `Slice(int, int)` method that makes it easy to do 2D slicing operations directly on the virtual 2D memory location, with the `Memory2D<T>` instance automatically adjusting the necessary parameters internally to shift its mapping on the right memory area(s) corresponding to the requested result.

Syntax

Here's how you can create a `Memory2D<T>` instance from a 2D array:

C#

```
int[,] array =
{
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};

Memory2D<int> memory = array;

// The memory directly maps the 2*3 array here

Memory2D<int> slice = memory.Slice(0, 1, 2, 2);

// We create a slice from row 0 and column 1, of size 2*2

int[,] copy = slice.ToArray();

// { 2, 3 }
// { 5, 6 }

// If on a supported runtime, we can also slice using a range

Memory2D<int> test = memory[.., ..2];

// { 1, 2 }
// { 4, 5 }
// { 7, 8 }

Span2D<int> span = memory.Span;
```

```
// We can use the span to perform operations on the underlying
// data for the memory instance. All the available APIs are
// documented in the docs about the Span2D<T> type.
```

ReadOnlyMemory2D<T>

The [ReadOnlyMemory2D<T>](#) is to the `Memory2D<T>` type what `ReadOnlyMemory<T>` is to `Memory<T>`. It exposes the same exact functionalities (minus the APIs that involve modifying the contents of the wrapped memory area) and provides a read-only view to arbitrary 2D memory locations. For more info on how this type works, you can refer to the paragraph on the `Memory2D<T>` type above.

Examples

You can find more examples in the [unit tests](#) [↗](#).

SpanOwner<T>

Article • 11/07/2024

The [SpanOwner<T>](#) is a stack-only buffer type that rents buffers from a shared memory pool. It essentially mirrors the functionality of [MemoryOwner<T>](#), but as a `ref struct` type. This is particularly useful for short-lived buffers that are only used in synchronous code (that don't require [Memory<T>](#) instances), as well as code running in a tight loop, as creating `SpanOwner<T>` values will not require memory allocations at all.

Platform APIs: [SpanOwner<T>](#), [MemoryOwner<T>](#)

Syntax

The same core features of `MemoryOwner<T>` apply to this type as well, with the exception of it being a stack-only `struct`, and the fact that it lacks the [IMemoryOwner<T>](#) interface implementation, as well as the `Memory<T>` property. The syntax is virtually identical to that used with `MemoryOwner<T>` as well, except for the differences mentioned above.

As an example, suppose we have a method where we need to allocate a temporary buffer of a specified size (let's call this value `length`), and then use it to perform some work. A first, inefficient version might look like this:

C#

```
byte[] buffer = new byte[length];  
  
// Use buffer here
```

This is not ideal, as we're allocating a new buffer every time this code is used, and then throwing it away immediately (as mentioned in the `MemoryOwner<T>` docs as well), which puts more pressure on the garbage collector. We can optimize the code above by using [ArrayPool<T>](#):

C#

```
// Using directive to access the ArrayPool<T> type  
using System.Buffers;  
  
int[] buffer = ArrayPool<int>.Shared.Rent(length);  
  
try
```

```

{
    // Slice the span, as it might be larger than the requested size
    Span<int> span = buffer.AsSpan(0, length);

    // Use the span here
}
finally
{
    ArrayPool<int>.Shared.Return(buffer);
}

```

The code above rents a buffer from an array pool, but it's more verbose and error-prone: we need to be careful with the `try/finally` block to make sure to always return the rented buffer to the pool. We can rewrite it by using the `SpanOwner<T>` type, like so:

```

C#

// Be sure to include this using at the top of the file:
using Microsoft.Toolkit.HighPerformance.Buffers;

using SpanOwner<int> buffer = SpanOwner<int>.Allocate(length);

Span<int> span = buffer.Span;

// Use the span here, no slicing necessary

```

The `SpanOwner<T>` instance will internally rent an array, and will take care of returning it to the pool when it goes out of scope. We no longer need to use a `try/finally` block either, as the C# compiler will add that automatically when expanding that `using` statement. As such, the `SpanOwner<T>` type can be seen as a lightweight wrapper around the `ArrayPool<T>` APIs, which makes them both more compact and easier to use, reducing the amount of code that needs to be written to properly rent and dispose short lived buffers. You can see how using `SpanOwner<T>` makes the code much shorter and more straightforward.

ⓘ Note

As this is a stack-only type, it relies on the duck-typed `IDisposable` pattern introduced with C# 8. That is shown in the sample above: the `SpanOwner<T>` type is being used within a `using` block despite the fact that the type doesn't implement the `IDisposable` interface at all, and it's also never boxed. The functionality is just the same: as soon as the buffer goes out of scope, it is automatically disposed. The APIs in `SpanOwner{T}` rely on this pattern for extra performance: they assume that the underlying buffer will never be disposed as long as the `SpanOwner<T>` type is in

scope, and they don't perform the additional checks that are done in `MemoryOwner<T>` to ensure that the buffer is in fact still available before returning a `Memory<T>` or `Span<T>` instance from it. As such, this type should always be used with a `using` block or expression. Not doing so will cause the underlying buffer not to be returned to the shared pool. Technically the same can also be achieved by manually calling `Dispose` on the `SpanOwner<T>` type (which doesn't require C# 8), but that is error prone and hence not recommended.

Examples

You can find more examples in the [unit tests](#) [↗].

MemoryOwner<T>

Article • 11/07/2024

The `MemoryOwner<T>` is a buffer type implementing `IMemoryOwner<T>`, an embedded length property and a series of performance oriented APIs. It is essentially a lightweight wrapper around the `ArrayPool<T>` type, with some additional helper utilities.

Platform APIs: `MemoryOwner<T>`, `AllocationMode`

How it works

`MemoryOwner<T>` has the following main features:

- One of the main issues of arrays returned by the `ArrayPool<T>` APIs and of the `IMemoryOwner<T>` instances returned by the `MemoryPool<T>` APIs is that the size specified by the user is only being used as a *minimum* size: the actual size of the returned buffers might actually be greater. `MemoryOwner<T>` solves this by also storing the original requested size, so that `Memory<T>` and `Span<T>` instances retrieved from it will never need to be manually sliced.
- When using `IMemoryOwner<T>`, getting a `Span<T>` for the underlying buffer requires first to get a `Memory<T>` instance, and then a `Span<T>`. This is fairly expensive, and often unnecessary, as the intermediate `Memory<T>` might actually not be needed at all. `MemoryOwner<T>` instead has an additional `Span` property which is extremely lightweight, as it directly wraps the internal `T[]` array being rented from the pool.
- Buffers rented from the pool are not cleared by default, which means that if they were not cleared when being previous returned to the pool, they might contain garbage data. Normally, users are required to clear these rented buffers manually, which can be verbose especially when done frequently. `MemoryOwner<T>` has a more flexible approach to this, through the `Allocate(int, AllocationMode)` API. This method not only allocates a new instance of exactly the requested size, but can also be used to specify which allocation mode to use: either the same one as `ArrayPool<T>`, or one that automatically clears the rented buffer.
- There are cases where a buffer might be rented with a greater size than what is actually needed, and then resized afterwards. This would normally require users to rent a new buffer and copy the region of interest from the old buffer. Instead, `MemoryOwner<T>` exposes a `Slice(int, int)` API that simply return a new instance wrapping the specified area of interest. This allows to skip renting a new buffer and copying the items entirely.

Syntax

Here is an example of how to rent a buffer and retrieve a `Memory<T>` instance:

C#

```
// Be sure to include this using at the top of the file:
using Microsoft.Toolkit.HighPerformance.Buffers;

using (MemoryOwner<int> buffer = MemoryOwner<int>.Allocate(42))
{
    // Both memory and span have exactly 42 items
    Memory<int> memory = buffer.Memory;
    Span<int> span = buffer.Span;

    // Writing to the span modifies the underlying buffer
    span[0] = 42;
}
```

In this example, we used a `using` block to declare the `MemoryOwner<T>` buffer: this is particularly useful as the underlying array will automatically be returned to the pool at the end of the block. If instead we don't have direct control over the lifetime of a `MemoryOwner<T>` instance, the buffer will simply be returned to the pool when the object is finalized by the garbage collector. In both cases, rented buffers will always be correctly returned to the shared pool.

When should this be used?

`MemoryOwner<T>` can be used as a general purpose buffer type, which has the advantage of minimizing the number of allocations done over time, as it internally reuses the same arrays from a shared pool. A common use case is to replace `new T[]` array allocations, especially when doing repeated operations that either require a temporary buffer to work on, or that produce a buffer as a result.

Suppose we have a dataset consisting of a series of binary files, and that we need to read all these files and process them in some way. To properly separate our code, we might end up writing a method that simply reads one binary file, which might look like this:

C#

```
public static byte[] GetBytesFromFile(string path)
{
    using Stream stream = File.OpenRead(path);
```

```

    byte[] buffer = new byte[(int)stream.Length];

    stream.Read(buffer, 0, buffer.Length);

    return buffer;
}

```

Note that `new byte[]` expression. If we read a large number of files, we'll end up allocating a lot of new arrays, which will put a lot of pressure over the garbage collector. We might want to refactor this code using buffers rented from a pool, like so:

C#

```

public static (byte[] Buffer, int Length) GetBytesFromFile(string path)
{
    using Stream stream = File.OpenRead(path);

    byte[] buffer = ArrayPool<T>.Shared.Rent((int)stream.Length);

    stream.Read(buffer, 0, (int)stream.Length);

    return (buffer, (int)stream.Length);
}

```

Using this approach, buffers are now rented from a pool, which means that in most cases we're able to skip an allocation. Additionally, since rented buffers are not cleared by default, we can also save the time needed to fill them with zeros, which gives us another small performance improvement. In the example above, loading 1000 files would bring the total allocation size from around 1MB down to just 1024 bytes - just a single buffer would effectively be allocated, and then reused automatically.

There are two main issues with the code above:

- `ArrayPool<T>` might return buffers that have a size greater than the requested one. To work around this issue, we need to return a tuple which also indicates the actual used size into our rented buffer.
- By simply returning an array, we need to be extra careful to properly track its lifetime and to return it to the appropriate pool. We might work around this issue by using `MemoryPool<T>` instead and by returning an `IMemoryOwner<T>` instance, but we still have the problem of rented buffers having a greater size than what we need. Additionally, `IMemoryOwner<T>` has some overhead when retrieving a `Span<T>` to work on, due to it being an interface, and the fact that we always need to get a `Memory<T>` instance first, and then a `Span<T>`.

To solve both these issues, we can refactor this code again by using `MemoryOwner<T>`:

C#

```
public static MemoryOwner<byte> GetBytesFromFile(string path)
{
    using Stream stream = File.OpenRead(path);

    MemoryOwner<byte> buffer =
        MemoryOwner<byte>.Allocate((int)stream.Length);

    stream.Read(buffer.Span);

    return buffer;
}
```

The returned `IMemoryOwner<byte>` instance will take care of disposing the underlying buffer and returning it to the pool when its `IDisposable.Dispose` method is invoked. We can use it to get a `Memory<T>` or `Span<T>` instance to interact with the loaded data, and then dispose the instance when we no longer need it. Additionally, all the `MemoryOwner<T>` properties (like `MemoryOwner<T>.Span`) respect the initial requested size we used, so we no longer need to manually keep track of the effective size within the rented buffer.

Examples

You can find more examples in the [unit tests](#) [↗].

StringPool

Article • 11/07/2024

The `StringPool` type implements a configurable pool for `string` instances. This can be used to minimize allocations when creating multiple `string` instances from buffers of `char` or `byte` values. It provides a mechanism that is somewhat similar to [string interning](#), with the main differences being that the pool is configurable, can be reset, is implemented with a best-effort policy and doesn't require to pre-instantiate `string` objects, so it can save the initial allocations as well when working on temporary buffers.

Platform APIs: [StringPool](#)

Syntax

The main entry point for `StringPool` is its `GetOrAdd(ReadOnlySpan<char>)` API, which returns a `string` instance matching the contents of the input `ReadOnlySpan<char>`, possibly getting the returned object from the internal pool.

As an example, imagine we have an input `string` representing the URL of a given web request, and we want to also retrieve a `string` with just the host name. If we get a large number of requests possibly for a small number of hosts, we might want to cache those `string` instances, we can do so by using the `StringPool` type as follows:

C#

```
public static string GetHost(string url)
{
    // We assume the input might start either with eg. https:// (or other
    // prefix),
    // or directly with the host name. Furthermore, we also assume that the
    // input
    // URL will always have a '/' character right after the host name.
    // For instance:
    "https://learn.microsoft.com/dotnet/api/system.string.intern".
    int
        prefixOffset = url.AsSpan().IndexOf(stackalloc char[] { ':', '/',
        '/' }),
        startIndex = prefixOffset == -1 ? 0 : prefixOffset + 3,
        endIndex = url.AsSpan(startIndex).IndexOf('/');

    // In this example, it would be "learn.microsoft.com"
    ReadOnlySpan<char> span = url.AsSpan(startIndex, endIndex);
```

```
    return StringPool.Shared.GetOrAdd(span);  
}
```

The method above does no allocations at all if the requested `string` is already present in the cache, as the lookup is done with just a `ReadOnlySpan<char>` as input, representing a view on the input URL `string`.

The `StringPool` type can also be useful when parsing raw requests using a different encoding, for instance UTF8. There is a `GetOrAdd` overload that takes an input `ReadOnlySpan<byte>` and an [Encoding](#) instance, which uses a temporary buffer rented from the pool to retrieve a `ReadOnlySpan<char>` value to use for the lookup. This again can greatly reduce the number of allocations depending on the specific use case scenario.

Examples

You can find more examples in the [unit tests](#) [↗](#).

ParallelHelper

Article • 11/07/2024

The [ParallelHelper](#) contains high performance APIs to work with parallel code. It contains performance oriented methods that can be used to quickly setup and execute parallel operations over a given data set or iteration range or area.

Platform APIs: [ParallelHelper](#), [IAction](#), [IAction2D](#), [IRefAction<T>](#), [IInAction<T><T>](#)

How it works

`ParallelHelper` type is built around three main concepts:

- It performs automatic batching over the target iteration range. This means that it automatically schedules the right number of working units based on the number of available CPU cores. This is done to reduce the overhead of invoking the parallel callback once for every single parallel iteration.
- It heavily leverages the way generic types are implemented in C#, and uses `struct` types implementing specific interfaces instead of delegates like [Action<T>](#). This is done so that the JIT compiler will be able to "see" each individual callback type being used, which allows it to inline the callback entirely, when possible. This can greatly reduce the overhead of each parallel iteration, especially when using very small callbacks, which would have a trivial cost with respect to the delegate invocation alone. Additionally, using a `struct` type as callback requires developers to manually handle variables that are being captured in the closure, which prevents accidental captures of the `this` pointer from instance methods and other values that could considerably slowdown each callback invocation. This is the same approach that is used in other performance-oriented libraries such as [ImageSharp](#) [↗](#).
- It exposes 4 types of APIs that represent 4 different types of iterations: 1D and 2D loops, items iteration with side effect and items iteration without side effect. Each type of action has a corresponding `interface` type that needs to be applied to the `struct` callbacks being passed to the `ParallelHelper` APIs: these are [IAction](#), [IAction2D](#), [IRefAction<T>](#) and [IInAction<T><T>](#). This helps developers to write code that is clearer regarding its intent, and allows the APIs to perform further optimizations internally.

Syntax

Let's say we're interested in processing all the items in some `float[]` array, and to multiply each of them by `2`. In this case we don't need to capture any variables: we can just use the `IRefAction<T>` interface and `ParallelHelper` will load each item to feed to our callback automatically. All that's needed is to define our callback, that will receive a `ref float` argument and perform the necessary operation:

C#

```
// Be sure to include this using at the top of the file:
using Microsoft.Toolkit.HighPerformance.Helpers;

// First declare the struct callback
public readonly struct ByTwoMultiplier : IRefAction<float>
{
    public void Invoke(ref float x) => x *= 2;
}

// Create an array and run the callback
float[] array = new float[10000];

ParallelHelper.ForEach<float, ByTwoMultiplier>(array);
```

With the `ForEach` API, we don't need to specify the iteration ranges: `ParallelHelper` will batch the collection and process each input item automatically. Furthermore, in this specific example we didn't even have to pass our `struct` as an argument: since it didn't contain any fields we needed to initialize, we could just specify its type as a type argument when invoking `ParallelHelper.ForEach`: that API will then create a new instance of that `struct` on its own, and use that to process the various items.

To introduce the concept of closures, suppose we want to multiply the array elements by a value that is specified at runtime. To do so, we need to "capture" that value in our callback `struct` type. We can do that like so:

C#

```
public readonly struct ItemsMultiplier : IRefAction<float>
{
    private readonly float factor;

    public ItemsMultiplier(float factor)
    {
        this.factor = factor;
    }

    public void Invoke(ref float x) => x *= this.factor;
}

// ...
```

```
ParallelHelper.ForEach(array, new ItemsMultiplier(3.14f));
```

We can see that the `struct` now contains a field that represents the factor we want to use to multiply elements, instead of using a constant. And when invoking `ForEach`, we're explicitly creating an instance of our callback type, with the factor we're interested in. Furthermore, in this case the C# compiler is also able to automatically recognize the type arguments we're using, so we can omit them together from the method invocation.

This approach of creating fields for values we need to access from a callback lets us explicitly declare what values we want to capture, which helps makes the code more expressive. This is exactly the same thing that the C# compiler does behind the scenes when we declare a lambda function or local function that accesses some local variable as well.

Here is another example, this time using the `For` API to initialize all the items of an array in parallel. Note how this time we're capturing the target array directly, and we're using the `IAction` interface for our callback, which gives our method the current parallel iteration index as argument:

C#

```
public readonly struct ArrayInitializer : IAction
{
    private readonly int[] array;

    public ArrayInitializer(int[] array)
    {
        this.array = array;
    }

    public void Invoke(int i)
    {
        this.array[i] = i;
    }
}

// ...

ParallelHelper.For(0, array.Length, new ArrayInitializer(array));
```

⚠ Note

Since the callback types are `struct`-s, they're passed *by copy* to each thread running parallel, not by reference. This means that value types being stored as fields in a callback types will be copied as well. A good practice to remember that

detail and avoid errors is to mark the callback `struct` as `readonly`, so that the C# compiler will not let us modify the values of its fields. This only applies to *instance* fields of a value type: if a callback `struct` has a `static` field of any type, or a reference field, then that value will correctly be shared between parallel threads.

Methods

These are the 4 main APIs exposed by `ParallelHelper`, corresponding to the `IAction`, `IAction2D`, `IRefAction<T>` and `IInAction<T>` interfaces. The `ParallelHelper` type also exposes a number of overloads for these methods, that offer a number of ways to specify the iteration range(s), or the type of input callback. `For` and `For2D` work on `IAction` and `IAction2D` instances, and they are meant to be used when some parallel work needs to be done that doesn't necessarily map to an underlying collection that can be directly accessed with the indices of each parallel iteration. The `ForEach` overloads instead work on `IRefAction<T>` and `IInAction<T>` instances, and they can be used when the parallel iterations directly map to items in a collection that can be indexed directly. In this case they also abstract away the indexing logic, so that each parallel invocation only has to worry on the input item to work on, and not on how to retrieve that item as well.

Examples

You can find more examples in the [unit tests](#) [↗](#).

Ref<T> and ReadOnlyRef<T>

Article • 11/07/2024

Ref<T>

The `Ref<T>` is a stack-only type that can store a reference to a value of a specified type. It is semantically equivalent to a `ref T` value, with the difference that it can also be used as a type of field in another stack-only `struct` type. It can be used in place of proper `ref T` fields, which are currently not supported in C#.

Platform APIs: `Ref<T>`, `ReadOnlyRef<T>`

How it works

⚠ Note

Due to how it's implemented on different .NET Standard contracts, the `Ref<T>` has some minor differences on .NET Standard 1.4 and .NET Standard 2.1 (and equivalent .NET Core runtimes). In particular, on .NET Standard 1.4 it can only be used to reference fields *within an object*, instead of arbitrary values pointed by a managed reference. This is because the underlying APIs being used on .NET Standard 1.4 don't have built-in support for the `Span<T>` type.

Ref<T> on .NET Standard 1.4

As mentioned before, on .NET Standard 1.4, `Ref<T>` can only point to locations within a given `object`. Consider the following code:

C#

```
// Be sure to include this using at the top of the file:
using Microsoft.Toolkit.HighPerformance;

// Define a sample model
class MyModel
{
    public int Number = 42;
}

var model = new MyModel();
```

```
// Create a Ref<T> pointing to the MyModel.Number field
Ref<int> byRef = new Ref<T>(model, ref model.Number);

// Modify the field indirectly
byRef.Value++;

Console.WriteLine(model.Number); // Prints 43!
```

Warning

The `Ref<T>` constructor doesn't validate the input arguments, meaning that it is your responsibility to pass a valid `object` and a reference to a field within that object. Failing to do so might cause the `Ref<T>.Value` property to return an invalid reference.

`Ref<T>` on .NET Standard 2.1

On .NET Standard 2.1, `Ref<T>` can point to any `ref T` value:

C#

```
int number = 42;

// Create a Ref<int> pointing to 'number'
Ref<int> byRef1 = new Ref<int>(ref number);

// Modify 'number'
byRef1.Value++;

Console.WriteLine(number); // Prints 43!
```

Warning

This type comes with a few caveats and should be used carefully, as it can lead to runtime crashes if a `Ref<T>` instance is created with an invalid reference. In particular, you should only create a `Ref<T>` instance pointing to values that have a lifetime that is greater than that of the `Ref<T>` in use. Consider the following snippet:

C#

```
public static ref int GetDummyReference()
{
```

```
int number = 42;

Ref<int> byRef = new Ref<T>(ref number);

return ref byRef.Value;
}
```

This will compile and run fine, but the returned `ref int` will be invalid (as in, it will not point to a valid location) and could cause crashes if it's dereferenced. It is your responsibility to track the lifetime of values being referenced by new `Ref<T>` values.

⚠ Note

Although it is possible to create a `Ref<T>` value wrapping a `null` reference, by using the `default(Ref<T>)` expression, the `Ref<T>` type is not designed to be used with nullable references and does not include proper features to validate the internal reference. If you need to return a reference that can be set to `null`, use the `NullableRef<T>` and `NullableReadOnlyRef<T>` types.

ReadOnlyRef<T>

The `ReadOnlyRef<T>` is a stack-only type that mirrors `Ref<T>`, with the exception that its constructor takes an `in T` parameter (a readonly reference), instead of a `ref T` one. Similarly, its `Value` property has a `ref readonly T` return type instead of `ref T`.

Examples

You can find more examples in the [unit tests](#) [↗](#).

Windows Community Toolkit Documentation

Article • 11/07/2024

The Windows Community Toolkit is a collection of controls for [WinUI 2](#), [WinUI 3](#), and [Uno Platform](#)  developers. Simplifies and demonstrates common developer tasks building experiences for Windows with .NET.

The Windows Community Toolkit is available as a NuGet package for new or existing C# projects. Read the [Getting Started](#) page for all the details.

You can also preview the capabilities of the toolkit by downloading the [Windows Community Toolkit Gallery](#)  in the Windows Store.

Feel free to browse the documentation using the table of contents on the left side of this page.

Open Source

[Windows Community Toolkit Source Code](#) 

Built as an open source project hosted on GitHub with support from the community. It is part of the [.NET Foundation](#) .

The majority of the projects within the Windows Community Toolkit are licensed under a MIT license. Check the specific repo/NuGet packages for details on licensing and third-party usage notes.

Get started

Article • 09/07/2023

This article covers how to get started using the packages provided as part of the Windows Community Toolkit project.

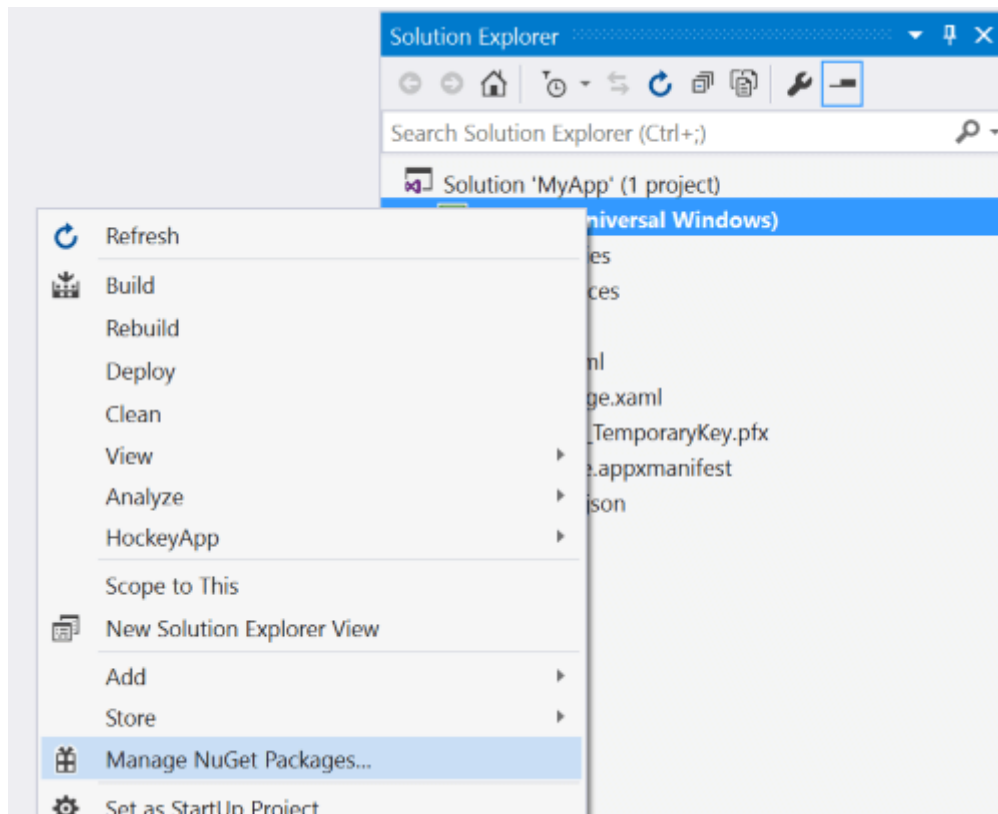
Adding the NuGet package(s)

The toolkit is available as a set of NuGet packages that can be added to any existing or new project using Visual Studio.

1. Open an existing project, or create a new project for the framework you're working with:
 - [UWP - WinUI2](#)
 - [WindowsAppSDK - WinUI3](#)
 - [Uno Platform](#) 
2. In the Solution Explorer panel, right click on your project name and select **Manage NuGet Packages**.

For *UWP/WinUI2* or *Uno.UI* based projects, search for **CommunityToolkit.Uwp**, and choose the desired NuGet Package from the list.

For *Windows App SDK/WinUI3* or *Uno.WinUI* based projects, search for **CommunityToolkit.WinUI** instead.



3. Choose the components that are most appropriate for your needs from the list and install.

You can see the available component packages listed on the left-hand table-of-contents.

4. Regardless of the package/framework you're using, namespaces will begin with `CommunityToolkit.WinUI`.

Other Resources

Download the [Windows Community Toolkit Gallery](#) from the Windows store to see the controls in an actual app.

Visit the [Windows Community Toolkit Github Repository](#) to see the current source code, what is coming next, and clone the repository. Community contributions are welcome! If you have an idea for a new feature, check out [Windows Community Toolkit Labs](#) for the latest experiments and process for proposing new features.

.NET Multi-platform App UI (.NET MAUI) Community Toolkit documentation

Article • 11/15/2023

The .NET MAUI Community Toolkit is a collection of reusable elements for application development with [.NET MAUI](#), including animations, behaviors, converters, effects, and helpers. It simplifies and demonstrates common developer tasks when building iOS, Android, macOS and WinUI applications using [.NET MAUI](#).

The MAUI Community Toolkit is available as a set of NuGet Packages for new or existing [.NET MAUI](#) projects.

You can also preview the capabilities of the toolkit by running the sample app available in the MAUI Community Toolkit repo.

Feel free to browse the documentation using the table of contents on the left side of this page.

Supported versions

The .NET MAUI Community Toolkit supports the platforms officially supported by [Microsoft](#):

- Android 5.0 (API 21) or higher.
- iOS 10 or higher.
- macOS 10.15 or higher, using Mac Catalyst.
- Windows 11 and Windows 10 version 1809 or higher, using [Windows UI Library \(WinUI\) 3](#).
- Tizen 7.0 or higher.

Get started

Follow the [Getting started guide](#) to install the **CommunityToolkit.Maui** NuGet packages into your existing or new [.NET MAUI](#) projects.

Open source

The .NET MAUI Community Toolkit is built as a set of open source projects hosted on GitHub by the community. Please feel free to join the team in contributing in any possible way that you can:

CommunityToolkit.Maui

- [GitHub repository](#) ↗
- [Report an issue](#) ↗
- [Documentation repository](#) ↗
- [Report an issue with the documentation](#) ↗

CommunityToolkit.Maui.Markup

- [GitHub repository](#) ↗
- [Report an issue](#) ↗
- [Documentation repository](#) ↗
- [Report an issue with the documentation](#) ↗